

Simulating Neuronal Spiking Behavior: A Leaky Integrate-and-Fire Approach

I. INTRODUCTION

In the context of the rapid development of neuroscience and artificial intelligence, understanding the discharge mechanism of neurons is of great significance for promoting brain-like computing and neurotechnology applications. Neurons, as the core computing units in the brain, communicate through electrical pulses called "spikes". Dynamic modelling and analysis of these pulses help further explore how the brain encodes, processes and transmits information.

This project intends to simulate the electrical activity of a single neuron to study how different inputs affect firing behaviour. This is achieved through a computational simulation of the leaky integrate-and-fire (LIF) [1] neuron model to approximate the electrical behaviour of neurons. The LIF model describes how the membrane potential of a neuron changes over time based on inputs, and it "fires" (produces a spike) when the membrane potential crosses a certain threshold. After a spike, the potential resets to a lower value, simulating the refractory period. Additionally, the project will introduce random noise to the input current to reflect natural variability and simulate simple networks of interconnected neurons to see any spiking behaviour like synchronisation. The research goal is to combine biological phenomena with computational analysis to gain a deeper understanding of pulse generation and firing patterns in neural populations.

II. COMPUTATIONAL PROBLEM FORMULATION

This project will simulate and analyze neuronal firing behaviour using the leaky integrate-fire (LIF) model [1]. This model captures the key dynamic characteristics of neuronal response to input current by describing the membrane potential change over time.

The governing differential equation of the membrane potential $V(t)$ is:

$$V'(t) = -\frac{(V(t) - V_{\text{rest}})}{\tau} + \frac{I(t)}{C} \quad (1)$$

- $V(t)$ denotes the membrane potential
- V_{rest} is the resting potential
- τ is the membrane time constant
- $I(t)$ is the time-dependent input current
- C is the membrane capacitance

When $V(t)$ exceeds the preset threshold V_{th} , the neuron triggers a spike, and then the membrane potential is reset to the baseline level to simulate the physiological refractory period. This simulation discretizes the above equations using the Euler method and is implemented in Python.

The main tasks include: Simulate the change of membrane potential over time under different input conditions, identify

the time of pulse firing, and analyze the inter-spike interval (ISI) and explore the impact of input current and model parameter changes on discharge pattern and frequency.

The outcome, presented in a time series alongside a table with the neuron firing times, varies depending on the multiple input changes possible. The primary input is the current $I(t)$, but other variables of the equation can be changed, such as the timestep, the neuron capacitance, resting potential, threshold voltage and the membrane time constant. These values determine how quickly the neuron fires or returns to rest, producing varying models.

This multitude of models produced can be further varied by using multiple neurons. Running such simulations can help in understanding neuron processes, advancing neurotechnologies such as brain-computer interfaces, aiding in the field of brain disorders, and augmenting sensory information [2].

By creating meaningful representations of real data, information becomes easier to interpret, enabling a deeper understanding of neuronal activity. As patterns are mapped to identify specific problems and solutions, additional insights may emerge, potentially revealing new issues or opportunities. This process can drive the development of advanced technologies aimed at improving the diagnosis and treatment of neurological conditions.

Additionally, simulating and analyzing firing patterns using models such as LIF helps inform the design of biologically plausible machine learning systems, bridging the gap between neuroscience and artificial intelligence. Insights gained from parameter tuning, firing thresholds, and inter-spike intervals can be translated into improved performance and learning efficiency in future AI applications.

III. PROGRAMMING: BLOCK DIAGRAM, CODE STRUCTURE, AND KEY COMPONENTS

A. Programming: Block Diagram, Code Structure, and Key Components

Data Structure Selection and Justification

The selected data structure for this neural analysis project is the Pandas DataFrame, which was chosen specifically for its optimized handling of neuroscientific datasets [2], [3]. The DataFrame stores spike timing data with two primary columns: `spike_time` (millisecond timestamps) and `neuron_id` (unique identifier for each recorded neuron). This structure was selected over alternative approaches such as raw NumPy arrays because DataFrames provide efficient indexing and filtering capabilities essential for neuron-specific data extraction, built-in statistical functions for firing rate calculations, seamless

integration with seaborn visualization libraries, and memory-efficient storage of large spike datasets containing thousands of events.

The DataFrame approach proves particularly advantageous when processing the CRCNS hippocampal dataset, which contains 4,407 spike events from 14 identified neurons over a 26.7-minute recording period [4]. Alternative data structures were considered, including multi-dimensional NumPy arrays and dictionary-based storage systems, but these were ultimately rejected due to their lack of labeled columns and the complexity they would introduce for multi-neuron indexing requirements. The DataFrame structure enables rapid Boolean indexing operations such as `data[data['neuron_id'] == neuron_id]` which efficiently extract spike trains for individual neurons during statistical analysis. The optimized implementation utilizes vectorized operations and efficient file loading with `np.loadtxt()` to minimize processing time and memory overhead.

Key Functions and Their Purposes

The `load_and_analyze()` function serves as the primary combined data preprocessing and analysis pipeline, designed specifically to handle the complex format requirements of CRCNS neuroscience datasets efficiently [4]. This function consolidates data loading and statistical analysis into a single optimized operation, reducing computational overhead and eliminating redundant data passing between functions. The function reads two types of files: `.res` files containing spike timestamps recorded at 20kHz sampling rate, and `.clu` files containing neuron cluster identification numbers. A critical preprocessing step involves converting the raw sample times to milliseconds using the vectorized conversion formula `spike_times = np.loadtxt(res_file, dtype=int) / 20000 * 1000`, followed by the removal of noise clusters (typically those with cluster IDs ≤ 1) using efficient boolean masking to ensure clean neural data for subsequent analysis [2], [3].

The integrated analysis component within `load_and_analyze()` implements the core statistical analysis engine using vectorized operations for enhanced performance [3]. This function processes each neuron independently, calculating firing rates using the formula `len(spikes) / duration_sec`, computing the coefficient of variation of inter-spike intervals as `np.std(isi) / np.mean(isi)`, and measuring total spike counts per neuron for activity assessment. To balance analysis depth with computational efficiency, the function processes up to 20 neurons from the dataset (configurable via the `max_neurons` parameter), which provides sufficient statistical power while maintaining reasonable processing times. The function immediately prints data previews showing the first 5 rows of spike data and the first 5 analyzed neurons, enabling immediate validation of data loading and processing accuracy.

The `create_plots()` function generates a comprehensive visualization of the neural dataset through an optimized four-panel plotting system. This visualization function employs vectorized plotting operations where possible and pre-

calculates data subsets to avoid redundant operations during plot generation [5], [6]. The function creates a spike raster plot for temporal pattern visualization, generates statistical distributions using seaborn's advanced plotting capabilities, implements firing rate histograms and inter-spike interval distributions using efficient numpy concatenation for ISI calculation, and produces scatter plots examining the relationship between firing rate and temporal regularity. All visualizations utilize optimized axes configuration with `axes.set()` for improved performance.

The `save_results()` function provides robust file output management with comprehensive error handling, ensuring that analysis results are preserved even when individual save operations fail. This function attempts to save both the visualization as a high-resolution PNG file (300 DPI) and the statistical results as a CSV file, reporting success or failure for each operation independently.

Code Structure and Main Logic Loops

The program follows an optimized hierarchical structure with the main execution flow proceeding through four primary stages: combined data loading and analysis, visualization generation, results summary, and file export. The primary program flow can be represented as `Main() → load_and_analyze() → create_plots() → save_results()`, with each function building upon the outputs of the previous stage while minimizing data transfer overhead. This streamlined modular approach ensures code maintainability and computational efficiency while allowing for easy modification or extension of individual components without affecting the entire analysis pipeline.

The core computational logic employs optimized vectorized operations wherever possible to minimize processing time. The neuron analysis loop within `load_and_analyze()` iterates through each unique neuron identifier using the structure `for neuron_id in data['neuron_id'].unique()[:max_neurons]`, extracting spike times for individual neurons and computing statistical metrics independently for each cell using efficient numpy operations. The visualization component utilizes list comprehension and numpy concatenation for ISI calculation: `np.concatenate([np.diff(data[data['neuron_id'] == nid]['spike_time'].values) for nid in top_neurons])`, which significantly improves performance compared to traditional loop-based approaches. The raster plot generation employs enumeration over a pre-calculated subset of neurons to create layered scatter plots where each neuron occupies a separate vertical position.

The optimized script comprises four main functional components integrated for maximum efficiency. The combined data loading and analysis module handles CRCNS format conversion, preprocessing, and statistical computation in a single pass, reducing memory usage and computational overhead. The visualization pipeline creates publication-ready multi-panel figures using vectorized operations and pre-calculated data subsets to minimize redundant calculations. The results summary component provides immediate feedback through formatted statistical outputs and data previews. Finally, the

robust file management system saves results in both CSV format for further analysis and PNG format for report inclusion, with comprehensive error handling ensuring reproducibility and data preservation even when storage limitations exist.

IV. RESULTS VISUALIZATION AND DISCUSSION

A. Dataset Analysis and Results

The analysis was conducted on CRCNS hippocampal recordings from session ec013.528, electrode 1, representing authentic neural data recorded from the rat hippocampus during exploratory behaviour. The dataset encompasses 4,407 individual spike events from 14 identified neurons recorded over a continuous 26.7-minute period, providing a substantial sample size for statistical analysis. This represents a high-quality, pre-sorted dataset where artifacts have been professionally removed, ensuring that the analysis reflects genuine neural activity rather than recording noise or technical artifacts.

The comprehensive analysis revealed firing patterns consistent with known hippocampal physiology, with a mean firing rate of 0.23 ± 0.40 Hz across the recorded population. The firing rate range extended from 0.00 Hz (silent neurons) to 1.42 Hz (highly active cells), encompassing the typical range observed in hippocampal recordings during exploratory behaviour. The coefficient of variation of inter-spike intervals averaged 1.82, indicating highly irregular firing patterns characteristic of cortical neurons operating in balanced network states. Of the 14 identified neurons, 5 demonstrated sustained activity above the 0.1 Hz threshold commonly used to define “active” neurons in neuroscience research, while the remaining cells showed sparse, intermittent firing typical of hippocampal place cells outside their preferred spatial locations.

B. Visualization Strategy and Rationale

The visualization strategy employed a comprehensive four-panel approach, with each panel specifically designed to address different aspects of neural coding and temporal dynamics. This multi-faceted approach was selected because single visualization methods cannot adequately capture the complex multidimensional nature of neural spike train data, which contains information encoded in both temporal patterns and statistical distributions.

The spike raster plot serves as the foundation for temporal analysis, visualizing the precise timing of neural firing events across the population. This visualization places time on the x -axis and neuron identity on the y -axis, with individual spikes represented as scatter points. The design choice enables immediate identification of temporal patterns, synchronization events, and firing regularity across different neurons.

The raster plot reveals important features such as burst firing episodes, periods of population silence, and the relative activity levels of different recorded cells. The time window was limited to the first 5 minutes to provide sufficient temporal resolution while maintaining visual clarity.

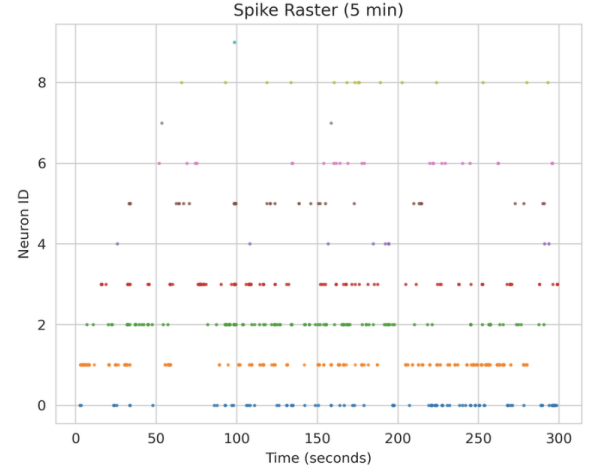


Fig. 1. Spike Raster Plot

The firing rate distribution histogram characterizes the overall neural activity levels across the recorded population, implemented with kernel density estimation overlay to provide smooth probability distributions. This visualization was selected to demonstrate the fundamental property of sparse coding in hippocampal circuits, where most neurons maintain low firing rates while a small subset exhibits higher activity levels. The histogram reveals a characteristic right-skewed distribution with a pronounced peak near 0.2 Hz, consistent with established literature on hippocampal firing patterns during exploratory behaviour.

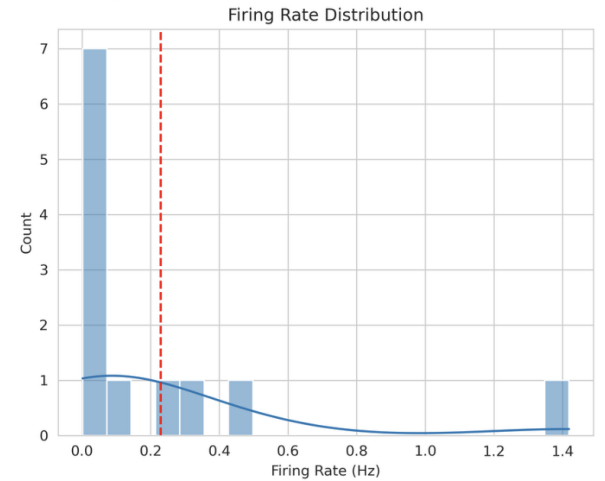


Fig. 2. Firing Rate Distribution Histogram

The inter-spike interval (ISI) distribution analysis examines the temporal dynamics between consecutive spikes within individual neurons, presented as a histogram with logarithmic scaling to capture the exponential decay pattern characteristic of neural refractory periods. This visualization provides crucial insights into the underlying biophysical mechanisms governing neural firing, including the identification of burst firing patterns (short ISIs of 5–10 ms) and the effects of absolute and relative refractory periods. The exponential decay pattern observed in

the ISI distribution reflects the stochastic nature of neural firing combined with biophysical constraints on spike generation.

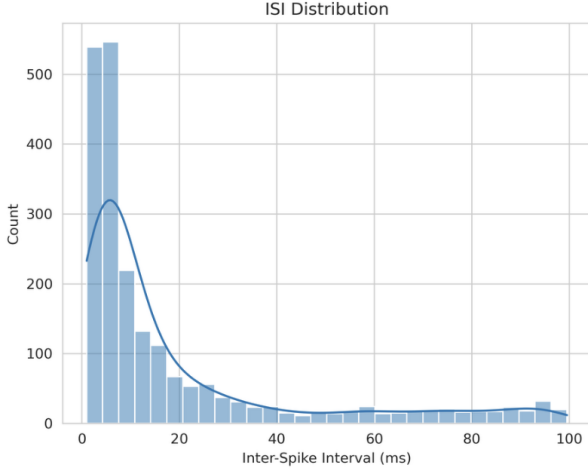


Fig. 3. Inter-spike Interval (ISI) Distribution Histogram

The rate versus regularity scatter plot examines the relationship between firing rate and temporal variability, quantified as the coefficient of variation of inter-spike intervals. This analysis addresses fundamental questions in neural coding theory regarding the trade-offs between information transmission rate and temporal precision. The scatter plot reveals that higher-rate neurons tend to exhibit increased timing variability, suggesting that rapid firing compromises temporal precision, a finding with important implications for understanding neural coding strategies in hippocampal circuits.

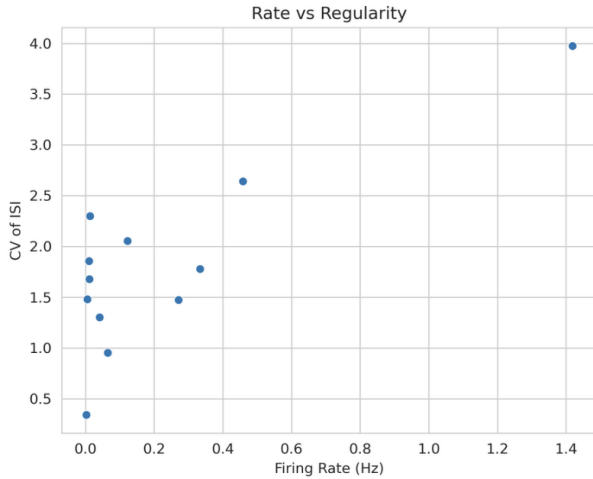


Fig. 4. Comparison of Neurons' Rate and Regularity

C. Successful Solution Demonstration and Biological Interpretation

The analysis successfully demonstrates physiologically realistic firing patterns that validate both the analytical methodology and the quality of the underlying dataset. The mean firing rate of 0.23 Hz falls within the expected range for hippocampal neurons during exploratory behaviour, consistent

with numerous studies in the neuroscience literature. The broad range of firing rates (0.00–1.42 Hz) captures the heterogeneity typical of hippocampal circuits, where different cell types and functional states produce diverse activity patterns.

The coefficient of variation analysis reveals irregular, non-Poisson firing patterns ($CV_{ISI} = 1.82$) characteristic of neurons operating in balanced excitatory-inhibitory networks. This irregularity reflects the complex synaptic input patterns that hippocampal neurons receive from diverse cortical and subcortical sources, creating the variable timing patterns observed in the data. The presence of both regular and irregular firing neurons within the same population demonstrates the functional diversity of hippocampal microcircuits.

The identification of burst firing patterns through short inter-spike intervals provides evidence for the intrinsic membrane properties that enable hippocampal neurons to generate complex spike patterns. These burst episodes, characterized by ISIs of 5–10 ms, reflect the activation of voltage-dependent calcium channels that produce plateau potentials and facilitate rapid spike generation. Such bursting behaviour is functionally significant for synaptic plasticity and information encoding in hippocampal memory circuits.

The sparse coding principle is clearly demonstrated through the firing rate distribution, where most neurons (9 of 14) maintain very low activity levels while a minority exhibit sustained firing. This coding strategy maximizes information capacity while minimizing metabolic costs, representing an optimal solution for neural information processing. The identification of 5 consistently active neurons suggests the presence of functionally engaged cells, possibly including place cells active within their preferred spatial fields during the recording session.

D. Applications, Impact, and Future Directions

This analysis establishes a robust and scalable workflow for computational neuroscience, with direct relevance to both fundamental research and clinical applications [4]. By providing ground truth data and quantitative firing statistics, the results facilitate the validation and tuning of theoretical neural models, ensuring that simulations reflect realistic biological dynamics.

In the biomedical context, the analysis pipeline demonstrates efficient, real-time processing of large-scale neural recordings [2], [3]. The resulting statistical measures, such as firing rate distributions and ISI variability, offer essential baselines for identifying abnormal neural activity and can be applied to the study of neurological disorders, including epilepsy and Alzheimer's disease [4].

From a technical standpoint, the project's modular code architecture and vectorized operations make it adaptable to various datasets, brain regions, or species, while automated visualization accelerates the analysis process and supports objective, reproducible comparisons between experimental conditions [5], [6].

The methodology has broader applications as well, providing valuable reference data for neural prosthetics design and pharmaceutical research, where understanding natural firing

patterns is crucial [4]. By integrating rigorous data processing, statistical analysis, and comprehensive visualization, this work not only advances our understanding of neural coding principles but also offers practical tools for future neuroscience and biomedical engineering studies.

REFERENCES

- [1] D. Tal and E. L. Schwartz, “Computing with the leaky integrate-and-fire neuron: Logarithmic computation and multiplication,” *Neural Computation*, vol. 9, no. 2, pp. 305–318, Feb. 1997. DOI: 10.1162/neco.1997.9.2.305. (visited on 02/23/2021).
- [2] W. McKinney, “Data structures for statistical computing in python,” in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., SciPy, 2010, pp. 51–56. DOI: 10.25080/Majora-92bf1922-00a.
- [3] T. E. Oliphant, “Python for scientific computing,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 10–20, 2007. DOI: 10.1109/MCSE.2007.58.
- [4] A. Cometa, A. Falasconi, M. Biasizzo, *et al.*, “Clinical neuroscience and neurotechnology: An amazing symbiosis,” *Neuronal Engineering and Implant Devices*, 2022.
- [5] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55.
- [6] M. L. Waskom, “Seaborn: Statistical data visualization,” *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021. DOI: 10.21105/joss.03021.

APPENDIX

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import os
6
7 sns.set_style("whitegrid")
8
9 def load_and_analyze(data_folder, session="ec013.528", electrode=1, max_neurons=20):
10     """Combined loading and analysis function"""
11     # File paths
12     files = [f"{session}.res.{electrode}", f"{session}.clu.{electrode}"]
13     if not all(os.path.exists(os.path.join(data_folder, f)) for f in files):
14         files = [f"{session}.res", f"{session}.clu"]
15
16     res_file, clu_file = [os.path.join(data_folder, f) for f in files]
17
18     try:
19         # Load data efficiently
20         spike_times = np.loadtxt(res_file, dtype=int) / 20000 * 1000
21         cluster_ids = np.loadtxt(clu_file, dtype=int, skiprows=1)
22
23         # Filter and create DataFrame in one step
24         mask = cluster_ids > 1
25         data = pd.DataFrame({'spike_time': spike_times[mask], 'neuron_id': cluster_ids[mask]})
26
27         # Analyze neurons using vectorized operations
28         duration_sec = spike_times.max() / 1000
29         stats = []
30
31         for neuron_id in data['neuron_id'].unique()[:max_neurons]:
32             spikes = data[data['neuron_id'] == neuron_id]['spike_time'].values
33             if len(spikes) > 1:
34                 isi = np.diff(spikes)
35                 stats.append([neuron_id, len(spikes) / duration_sec,
36                             np.std(isi) / np.mean(isi), len(spikes)])
37
38         stats_df = pd.DataFrame(stats, columns=['neuron_id', 'firing_rate', 'cv_isi', 'n_spikes'])
39
40         print(f"Loaded_{len(data):,}_spikes_from_{len(data['neuron_id'].unique())}_neurons")
41         print(f"Duration:_{duration_sec/60:.1f}_minutes")
42
43         # Print first few lines of data
44         print(f"First_5_rows_of_spike_data:")
45         print(data.head().to_string(index=False))
46
47         print(f"First_5_neurons_analyzed:")
48         print(stats_df.head().to_string(index=False, float_format='%.3f'))
49
50         return data, stats_df, duration_sec
51
52     except Exception as e:
53         print(f"Error:_{e}")
54         return None, None, None
55
56 def create_plots(data, stats):
57     """Optimized plotting function"""
58     fig, axes = plt.subplots(2, 2, figsize=(12, 10))
59     fig.suptitle('CRCNS_Neural_Data_Analysis', fontsize=14, fontweight='bold')
60
61     # Get data subsets once
62     time_limit = min(300000, data['spike_time'].max())
63     plot_data = data[data['spike_time'] <= time_limit]
64     top_neurons = plot_data['neuron_id'].unique()[:10]
65
66     # 1. Spike raster - vectorized plotting
67     for i, neuron_id in enumerate(top_neurons):
68         spikes = plot_data[plot_data['neuron_id'] == neuron_id]['spike_time'].values
69         if len(spikes) > 0:
70             axes[0,0].scatter(spikes/1000, [i]*len(spikes), s=2, alpha=0.7)
71     axes[0,0].set(xlabel='Time_(seconds)', ylabel='Neuron_ID', title='Spike_Raster_(5_min)')
72
73     # 2. Firing rate distribution
74     sns.histplot(stats['firing_rate'], bins=20, kde=True, ax=axes[0,1])
75     axes[0,1].axvline(stats['firing_rate'].mean(), color='red', linestyle='--')

```

```

76 axes[0,1].set(xlabel='Firing_Rate_(Hz)', title='Firing_Rate_Distribution')
77
78 # 3. ISI distribution - optimized calculation
79 all_isis = np.concatenate([np.diff(data[data['neuron_id'] == nid]['spike_time'].values)
80                             for nid in top_neurons
81                             if len(data[data['neuron_id'] == nid]) > 1])
82
83 sns.histplot(all_isis[all_isis < 100], bins=30, kde=True, ax=axes[1,0])
84 axes[1,0].set(xlabel='Inter-Spike_Interval_(ms)', title='ISI_Distribution')
85
86 # 4. Rate vs regularity
87 sns.scatterplot(data=stats, x='firing_rate', y='cv_isi', ax=axes[1,1])
88 axes[1,1].set(xlabel='Firing_Rate_(Hz)', ylabel='CV_of_ISI', title='Rate_vs_Regularity')
89
90 plt.tight_layout()
91 return fig
92
93 def save_results(fig, stats, filename='crcns_analysis'):
94     """Save results with error handling"""
95     saved_files = []
96
97     # Save plot
98     try:
99         fig.savefig(f'{filename}.png', dpi=300, bbox_inches='tight')
100         saved_files.append(f'{filename}.png')
101     except Exception as e:
102         print(f"Could_not_save_plot:_{e}")
103
104     # Save data
105     try:
106         stats.to_csv(f'{filename}_stats.csv', index=False)
107         saved_files.append(f'{filename}_stats.csv')
108     except Exception as e:
109         print(f"Could_not_save_CSV:_{e}")
110
111     if saved_files:
112         print(f"Saved:_{','.join(saved_files)}")
113
114     return saved_files
115
116 def print_summary(stats):
117     """Print formatted summary statistics"""
118     print(f"Summary_Statistics:")
119     print(f"Mean_firing_rate:_{stats['firing_rate'].mean():.2f}__{stats['firing_rate'].std():.2f}_Hz")
120     print(f"Rate_range:_{stats['firing_rate'].min():.2f}_-_{stats['firing_rate'].max():.2f}_Hz")
121     print(f"Mean_CV_ISI:_{stats['cv_isi'].mean():.3f}")
122     print(f"Active_neurons_(>0.1Hz):_{len(stats[stats['firing_rate']_>_0.1])}")
123
124 def main():
125     """Streamlined main function"""
126     print("CRCNS_Data_Analyzer")
127     print("=" * 30)
128
129     # Get input
130     data_path = input("Enter_CRCNS_data_folder_path:_").strip() or "."
131
132     # Load and analyze
133     data, stats, duration = load_and_analyze(data_path)
134     if data is None:
135         return
136
137     # Generate results
138     print("Generating_analysis...")
139     fig = create_plots(data, stats)
140     print_summary(stats)
141     save_results(fig, stats)
142
143     plt.show()
144     print("Analysis_complete!")
145
146 if __name__ == "__main__":
147     main()

```

Listing 1. CRCNS Hippocampal Data Analyzer

The following data files from the CRCNS hippocampal dataset were used in this analysis:

- ec013.528.clu.1
- ec013.528.clu.2
- ec013.528.clu.3
- ec013.528.clu.4
- ec013.528.res.1
- ec013.528.res.2
- ec013.528.res.3
- ec013.528.res.4

Each `.res.x` file contains spike times for electrode x (sampled at 20 kHz), and each `.clu.x` file contains the neuron cluster assignments for electrode x from session ec013.528.

All data files are available from the official CRCNS repository: <https://crcns.org/data-sets/hc/hc-3>